

## Article

# AFL++: A Vulnerability Discovery and Reproduction Framework

Guofeng He <sup>1</sup>, Yichen Xin <sup>1</sup>, Xiuchuan Cheng <sup>1</sup> and Guangqiang Yin <sup>1,2,\*</sup>

<sup>1</sup> School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu 611730, China; 201812090812@std.uestc.edu.cn (G.H.); 202312081601@std.uestc.edu.cn (Y.X.); 202112281006@std.uestc.edu.cn (X.C.)

<sup>2</sup> Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518110, China

\* Correspondence: yingq@uestc.edu.cn

**Abstract:** Directed greybox fuzzing can mainly be used for vulnerability mining and vulnerability replication. However, there are still some issues with existing directional fuzzing tools. One is that after providing problematic changes or patches, it is not possible to quickly target and discover the problem. Secondly, it is difficult to break through the magic byte path, making it difficult to mine deep vulnerabilities. This article proposes a new vulnerability mining and repair framework: American Fuzz Lop Plus (AFL++). Firstly, we utilize alias analysis to enhance inter-procedural control flow graphs and redefine the distance calculation formula to obtain more accurate distances. Secondly, the Newton interpolation method is used for the energy initialization of each seed to prevent test cases from being filtered out due to low energy. A heuristic energy scheduling algorithm is proposed to judiciously schedule the energy of seeds. During the path exploration phase, by adjusting the seed energy, shorter-distance seeds quickly reach the target; with increasing time, seeds tend to explore deeper paths. We then represent the symbolic distance by the number of instructions passed to reach the target and investigate the shortest path search strategy to achieve path pruning, alleviating the problem of path explosion. Finally, based on the above methods, we implement the AFL++ prototype system, integrating directed greybox fuzzing with symbolic execution technology for vulnerability discovery. By interleaving directed symbolic execution and directed greybox fuzzing, the efficiency of vulnerability discovery and reproduction is effectively enhanced.

**Keywords:** fuzzing testing; targeted fuzzing; symbolic execution; energy scheduling; vulnerability discovery



**Citation:** He, G.; Xin, Y.; Cheng, X.; Yin, G. AFL++: A Vulnerability Discovery and Reproduction Framework. *Electronics* **2024**, *13*, 912. <https://doi.org/10.3390/electronics13050912>

Received: 5 January 2024

Revised: 22 February 2024

Accepted: 26 February 2024

Published: 27 February 2024



**Copyright:** © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Fuzzing [1] is an automated testing technique that inputs random, invalid, or exceptional data into a target program or system to trigger potential vulnerabilities or abnormal behaviors. Fuzz testing methods can be divided into blackbox, whitebox, and greybox. A magic byte [1] is a type of byte or sequence of bytes in binary files used to identify the file type or protocol. In the process of vulnerability discovery, magic bytes are commonly used to recognize specific file formats or data streams.

Fuzzing is currently a mainstream technique for detecting vulnerabilities [1–4]. By injecting random or anomalous inputs into a program under test (PUT) and then monitoring its anomalous behavior, vulnerabilities and attack surfaces can be uncovered where software is located. Directional fuzzing [5–8] can mainly be used for vulnerability mining and vulnerability recurrence. After a known vulnerability is suspected, directed greybox fuzzing has a better effect than classic fuzzing. Despite many years of development, many difficulties and unsolved problems remain [9–13]. One is that after a problematic change or patch is given, it cannot be quickly directed to the target to find the problem. The second is that breaking through the magic byte path is difficult, making it challenging to dig deep into vulnerabilities. Therefore, the research focus of this paper is to discuss the following

two issues: one is how to quickly locate the target area to improve the overall vulnerability mining efficiency, and the other is how to make the seeds that are farther away converge to the target point to increase the coverage of the target area. Finding loopholes in the deep path is difficult; the seeds cannot be quickly oriented, and the long-distance seeds are discarded.

Fuzzing is the prevailing technique in today's vulnerability detection landscape [1–4]. By injecting random or anomalous inputs into a program under test (PUT) and monitoring its abnormal behavior, fuzzing can reveal vulnerabilities and potential attack surfaces within software. Directed greybox fuzzing [5–8] is primarily employed for vulnerability discovery and reproduction. Particularly when known vulnerabilities are suspected, directed greybox fuzzing exhibits superior effectiveness compared to classic fuzzing. Despite several years of development, there are still numerous challenges and unresolved issues [9–13]. Firstly, after introducing changes or patches to address the identified issues, directed greybox fuzzing often struggles to rapidly pinpoint the target area for issue discovery. Secondly, difficulties arise in breaking through magic byte paths, resulting in challenges when attempting to explore deeper vulnerabilities. Therefore, the focal points of this research are to address the following two issues: firstly, how to swiftly locate the target area to enhance overall vulnerability discovery efficiency; and secondly, how to converge seeds from greater distances to the target point, thereby increasing coverage of the target area. The challenges to be addressed include the difficulty in discovering vulnerabilities in deeper paths, the inability to rapidly target seeds, and the tendency to discard seeds at greater distances from the target.

The innovative aspects of this research mainly include the following:

- (1) By optimizing the distance calculation formula and employing a heuristic energy scheduling algorithm, seeds not only possess more accurate distances but also, during continuous execution, trigger more paths and unearth additional vulnerabilities.
- (2) We introduce a distance-guided symbolic execution technique. When fuzzing fails to trigger new states for an extended period, symbolic execution is initiated. A shortest-path search algorithm is employed to mitigate path explosion and reduce overhead. Seeds generated through symbolic execution are incorporated into the fuzzing queue, and subsequent mutations are performed to generate higher-quality seeds that trigger new states, thereby enhancing the efficiency of vulnerability discovery.
- (3) We propose the American Fuzz Lop Plus (AFL++) framework for vulnerability discovery based on directed greybox fuzzing and symbolic execution. Comparative experiments with American Fuzz Lop (AFL) and American Fuzz Lop Go (AFLGo) on eight real open-source programs and the LAVA-M dataset demonstrate that AFL++ effectively increases code coverage and improves the efficiency of vulnerability discovery.

## 2. Related Work

### 2.1. Directed Greybox Fuzzing

In 2017, Marcel Böhme et al. [1] introduced AFLGo, a directed greybox fuzzing technique that utilized a simulated annealing power algorithm to allocate energy to seeds based on their distance to the target. In 2018, Hawkeye [9] improved upon AFLGo by enhancing the function call graph through implicit call analysis. In 2019, a novel energy scheduling algorithm, SCDF [10], was proposed, adjusting the energy based on seed coverage to dynamically compute the capability of a given statement sequence. DrillerGo [5], in 2019, combined driller and concolic techniques, enabling the fast exploration of input spaces through fuzzing, while concolic execution addressed complex path conditions. In 2020, SDHF [14] leveraged sequence-directed strategies and concolic execution to enhance the effectiveness of fuzzing. ParmeSan [15], also in 2020, dynamically constructed accurate control flow graphs, employing a two-tier directed fuzz strategy to effectively reach all specified targets. DeFuzz [16], in the same year, proposed a deep learning-guided directed greybox fuzzing approach for software vulnerability detection. In 2021, Kailong Zhu [17] addressed indirect jump relationships in directed greybox testing from a control flow graph

(CFG) perspective. Gwangmu Lee [18], in 2021, introduced constraint-guided directed greybox fuzzing, defining the constraints as a set of target positions and data conditions to guide seeds in meeting the specified constraints sequentially. In the same year, targeted fuzzing techniques based on regression, keypoints, and cooperative parallelism were also proposed [19–21]. In 2022, BEACON [22], featuring path pruning, was introduced. In contrast to increasing coverage, BEACON reduces overhead by pruning irrelevant branches, saving computational resources to enhance efficiency. WindRanger [23] promotes directed greybox fuzzing (DGF) by aligning execution paths with the deviation from target sites. TargetFuzz [24], using DART [16], guides DGF [25].

## 2.2. Symbolic Execution

In 2008, the introduction of KLEE [24] marked pioneering work in the field of symbolic execution, and many subsequent outstanding symbolic execution tools have largely been built upon the foundation laid by KLEE. In 2011, directed symbolic execution [26] explored the automated identification of program execution paths leading to specific targets. It employed single-direction symbolic execution (SDSE) to guide symbolic execution forward, combining concolic symbolic execution (CCBSE) and Mix-CCBSE in an iterative, alternating execution for forward symbolic execution. In 2020, Bugminer [27] integrated AFL with dynamic symbolic execution and machine learning to trigger potential, challenging-to-reach vulnerabilities within program binary files. Symcc [28], also in 2020, introduced a compiler-based approach allowing hooking within the compiler and instrumentation on the target code. In 2021, Symcc further proposed SymQEMU [29], a novel compiler-based symbolic execution technique for binary files. It is based on QEMU and modifies intermediate representation (IR) before the target program is transformed into machine code for the host architecture, enabling SymQEMU to be applied to binary files.

Existing fuzzing tools have many shortcomings. Firstly, in the current distance-guided fuzzing, the definition of distance is not accurate enough, which cannot accurately guide the target. Secondly, some indirect calls are ignored as they only directly call to obtain the control flow graph (CFG), resulting in an incomplete inter-procedural control flow graph (ICFG). Then, most targeted fuzzing tools only focus on the seeds of shorter paths, leading to the inability to trigger some deeper paths. Moreover, due to the unreasonable energy distribution and scheduling, a large number of seeds are directed toward redundant branches, resulting in low coverage of the target area and difficulty in mining deep vulnerabilities. Finally, symbolic execution cannot perfectly adapt to targeted fuzzing, and there are issues of path explosion due to full coverage.

## 3. The Proposed AFL++ Method

### 3.1. Distance-Guided Fuzzing Technique

#### 3.1.1. Optimization Algorithm for Basic Block Distance Calculation

Once the target is identified, the ideal scenario is that all paths reachable from the initial point to the target should be executed, meaning seeds with a non-infinite distance are accessible. The concept employed by AFL is to select seeds with the shortest execution time and the smallest bit size. However, this approach is not entirely fair to other seeds, and consistently using this strategy does not significantly increase coverage in the target area, reducing the probability of discovering deeper vulnerabilities. Therefore, this paper optimizes the distance calculation algorithm, as shown in Equation (1). This optimization focuses on shorter-distance seeds while narrowing the gap between seeds with longer distances.

$$d_b(s, T_b) = \frac{\sum_{m \in \xi(s)} d_b(m, T_b)}{|\xi(s)| * (T_{bn} + \epsilon)} \quad (1)$$

Simultaneously, following this concept, distances to unreachable basic blocks are modified to  $\infty$ . For basic blocks with multiple targets, the distance is calculated according to the same formula. Here,  $d_b$  represents the distance from the seed to the target,  $T_{bn}$

represents the number of basic blocks traversed, and  $\epsilon$  is a very small number introduced to ensure the denominator is not zero.

The aim of this paper is to ensure that seeds capable of reaching the target point do so rapidly, allowing seeds associated with deeper paths to also have the opportunity for execution. Apart from providing higher energy to seeds with shorter distances, we allocate relatively equal energy to seeds with longer distances, bridging the gap between closer and more distant seeds. To achieve this, a reward factor is defined, where higher distances receive higher rewards, as illustrated in Equation (2). Here,  $c$  is a constant, typically set to 0.5 by default.

$$r_b(m, T_b) = c \sum_{m \in T_b} d_b(s, T_b)^{-2} \quad (2)$$

For the seed  $s$ , its set of basic blocks is  $\xi(s)$ . Then, the reward  $r_s$  is as follows:

$$r_s(s) = \sum_{m \in \xi(s)} r_b(m, T_b) \quad (3)$$

Next, the reward factor is added to the power factor to form the new power factor, where  $power\_factor = 2^{10 * p(s, T_b) - 5} + r_s$ . This combination effectively increases the weight of seeds with longer distances. Subsequently, seed mutations are selected based on this weight distribution, thereby enhancing the probability of discovering vulnerabilities and effectively increasing the coverage of target blocks to further improve the overall vulnerability discovery efficiency.

### 3.1.2. Energy Distribution Method Based on Newton Interpolation

In AFLGo, the initial energy allocation for seeds is based on factors such as execution time, seed size, and execution depth. However, due to discrete energy allocation, there exists an energy interval. Achieving rationalized energy distribution requires using function modeling to allocate energy to seeds rather than randomly assigning initial energy based on discrete values.

We employed the Newton interpolation method to fit multiple discrete values. The advantage of this method lies in the fact that when adding new energy point coordinates, only the relevant portion for the new points needs to be computed rather than recalculating the entire function from the beginning, making it more convenient.

Its implementation is as follows: Assume that the value of the  $n+1$  point relative polynomial function  $f(x)$  is known:  $(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))$ . The two-point Newton interpolation is  $f_1(x) = f(x_0) + b_1(x - x_0)$ , where  $f_1(x_0) = f(x_0)$ , and by substituting  $x_1$ , can solve  $b_1$ . The three-point Newton interpolation is  $(x - x_0)(x - x_1)$ , which can guarantee  $f_2(x_0) = f(x_0)$ ,  $f_2(x_1) = f(x_1)$ , and  $f_2(x) = f_1(x) + b_2(x - x_0)(x - x_1)$  is substituted into  $f_2(x_2) = f(x_2)$  to solve  $b_2$ .

Observing the characteristics of  $b_1$  and  $b_2$ , and repeating the above process continuously,  $b_n$  can be solved, and the fitting function can be obtained. The final difference quotient table is shown in Table 1.

**Table 1.** The difference quotient table.

| $x_k$ | $f(x_k)$ | 1st Order<br>Difference<br>Quotient | 2nd Order<br>Difference<br>Quotient | ... | n Order<br>Difference<br>Quotient |
|-------|----------|-------------------------------------|-------------------------------------|-----|-----------------------------------|
| $x_0$ | $f(x_0)$ |                                     |                                     |     |                                   |
| $x_1$ | $f(x_1)$ | $f[x_0, x_1]$                       |                                     |     |                                   |
| $x_2$ | $f(x_2)$ | $f[x_1, x_2]$                       | $f[x_0, x_1, x_2]$                  |     |                                   |
| $x_3$ | $f(x_3)$ | $f[x_2, x_3]$                       | $f[x_1, x_2, x_3]$                  |     |                                   |
| ...   | ...      | ...                                 | ...                                 | ... |                                   |
| $x_n$ | $f(x_n)$ | $f[x_{n-1}, x_n]$                   | $f[x_{n-2}, x_{n-1}, x_n]$          | ... | $f[x_0, x_1, \dots, x_n]$         |

The n-order difference quotient is calculated as follows:

$$f[x_0, x_1, \dots, x_n] = \frac{f[x_0, x_1, \dots, x_{n-1}] - f[x_1, x_2, \dots, x_n]}{x_0 - x_n} \quad (4)$$

Utilizing the interpolation algorithm mentioned above, the initial default energy reward value is set to 100. This initial value is set to prevent the issue of initially inputted test cases with very low energy from being filtered out, resulting in subsequent execution problems. This effectively lowers the requirements for inputs. The algorithm enables a more fine-grained, rationalized allocation of energy to seeds toward the target basic blocks, ensuring different seeds have distinct effectiveness. This enhancement contributes to the efficiency of vulnerability discovery in directed greybox fuzzing.

### 3.1.3. PSO-Based Heuristic Energy Scheduling Algorithm

In order to fully utilize the energy of all seeds reachable at the target, the entire seed queue can be treated as a population. Since the state of each execution result can be saved, the state of the entire population can be known. At this point, a direction can be specified to bring the entire population closer to the target state, enabling all seeds to reach the target point as indiscriminately as possible. Based on this idea, we chose the particle swarm optimization (PSO) algorithm to guide seeds toward the target point.

The optimal scheduling strategy is to ensure that seeds with shorter distances are allocated more energy, while also ensuring that seeds with greater depth receive a higher energy allocation. Therefore, the depth factor of seeds is considered, where the position, distance, depth, and energy of seeds are correspondingly crucial for implementing the algorithm.

#### Design of Fitness Function

In this optimization problem, each seed possesses initial energy. We employed normalized distance processing to integrate distance and depth into a unified objective.

$$Z = p_1 \frac{d(s, T_b) - \min D}{\max D - \min D} + p_2 [1 - \frac{\text{depth}_i - \min_{\text{depth}}}{\max_{\text{depth}} - \min_{\text{depth}}}] \quad (5)$$

$$\text{fit}_i = \max(\frac{1}{Z}) \quad (6)$$

where  $p_1 + p_2 = 1$  and  $p_1 > 0, p_2 > 0$  are expressed as the relative importance of the two factors of distance and depth. The reason for taking the inverse is to magnify the difference, which makes it easier to compare when making comparisons.

For Equation (5), assuming  $Z = f(x)$  and deriving the distance  $di$  and depth  $de$ , respectively, the following results can be obtained:

$$\begin{cases} f_{di}(di, de) = \frac{\partial Z}{\partial di} = \frac{p_1}{\max D - \min D} \\ f_{de}(di, de) = \frac{\partial Z}{\partial de} = \frac{p_2}{\max_{\text{depth}} - \min_{\text{depth}}} \end{cases} \quad (7)$$

Through Equation (7), it can be observed that  $Z$  is directly proportional to the distance and inversely proportional to the depth. Initially, define  $p_1 = 1, p_2 = 0$ . In the continuous loop of execution, continuously adjust the values of  $p_1$  and  $p_2$ , as shown in Equation (8), so that the center of gravity moves toward depth and coverage. The idea is that, initially, due to random inputs, depth does not hold a significant advantage. Therefore, distance is prioritized at the beginning, focusing on executing paths with shorter distances that can rapidly trigger target points while continuously mutating. As time progresses, most of the shorter-distance paths are likely to have been executed, leaving them repeating. Consequently, the strategy is modified to shift toward paths with greater depth.

$$\begin{cases} p_1 = 20^{-\frac{t}{t_x}} \\ p_2 = 1 - p_1 \end{cases} \quad (8)$$

### Velocity and Position Updates

The equation for calculating the speed of the seed in the update is as follows:

$$v_i(t+1) = \omega \times v_i(t) + c_1 \times rand() \times (pbest_i(t) - x_i(t)) + c_2 \times rand() \times (gbest_i(t) - x_i(t)) \quad (9)$$

where  $\omega$  is the inertia factor,  $v_i$  is the speed of the seed,  $rand()$  is a random number,  $x_i$  is the distance vector of the current execution seed,  $c_1, c_2$  are the learning factors, usually  $c_1 = c_2 = 2$ , which, in this paper, are defined as 1.49445, and  $pbest_i$  and  $gbest_i$  are two extreme values of the individual and group, respectively.

The ideal position indicates that the seed should be executed at the target point and which position should be executed in the next step to gradually approach it. The updated equation is as follows:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (10)$$

The default weight  $\omega$  adopts a linear decreasing strategy  $\omega^t = (\omega_{init} - \omega_{end})(G_k - g) / G_k + \omega_{end}$ , where  $G_k$  is the maximum number of iterations. However, since the fuzz test is always running, we modified the equation as follows:

$$\omega^t = (\omega_{init} - \omega_{end}) \times T + \omega_{end} \quad (11)$$

where  $\omega_{init}$  is the initial inertia weight, which is defined as 0.9 in this paper;  $\omega_{end}$  is the inertia weight when iterating to the maximum number of evolutions, which is defined as 0.4 in this paper;  $T$  is the current elapsed time ratio; and the total time is 48 h. In this study, different algorithms were selected according to different parameters to control the convergence of weights, and  $T_{exp}$  was selected by default.

$$T_{exp} = T_0 \times \alpha^k \quad (12)$$

When  $0.05 = \alpha^{k_x}$ , use equation X to obtain  $k_x = \log(0.05) / \log(\alpha)$ , and then obtain  $T_{exp} = \alpha^{\frac{t \log(0.05)}{T_x \log(\alpha)}}$ , which can be simplified to  $T_{exp} = 20^{-\frac{t}{T_x}}$ , so that the updated final weight formula is as follows:

$$\omega^t = (\omega_{init} - \omega_{end}) \times 20^{-\frac{t}{T_x}} + \omega_{end} \quad (13)$$

The initial weight is 0.9, so the seed can be better explored globally. As time passes, the seed slowly moves closer to the target position. In order to better achieve the optimum, the weight and the randomness are reduced. Therefore, in “foraging” for seeds, regardless of the distance, they will slowly approach the final goal.

### 3.2. Distance-Guided Symbolic Execution Technique

#### 3.2.1. Hybrid Symbolic Execution

The framework presented in this paper is based on the KLEE framework. KLEE requires symbolic inputs to run, so it uses the `klees_make_symbolic` function to treat variables as symbolic inputs and then performs symbolic execution. However, this approach has significant drawbacks. During execution, data generated by fuzzing may be used as symbolic input, and if the current symbolic value cannot satisfy the current constraints, both situations can lead to an inability to continue exploring new paths. Despite KLEE rewriting some system call functions, there are still unrecognized system functions. Moreover, many branches are prone to causing path explosions. Through hybrid execution, some branches can be reduced by using concrete values, improving the issue of path explosion. Therefore, hybrid execution is needed.

Firstly, define a struct for concrete input values and a boolean to identify whether it is a concrete value. The struct includes the field “LLVM::Value” to denote the type of concrete value. Then, obtain references to the LLVM nodes of all instructions, where the node is of the “Instruction” type and includes the opcode, instruction type, and basic

block information. Subsequently, rewrite part of the instructions by accessing the operands through the operand indices and performing calculations on the operands based on the operation. Use a “ValueMap” to maintain the symbolic state and record the constraint conditions of the execution path. During execution, first check whether it is a concrete value. If it is, directly substitute it into the calculation to simplify the expression, and mark the status of the current PHI node as completed. At each branch where the symbolic values are encountered, perform detection and calculation. If it is a concrete value, execute according to the result path of the concrete value. Otherwise, apply negation constraints to solve for new paths, ultimately achieving hybrid execution. The algorithm flow is illustrated in Algorithm 1.

---

**Algorithm 1:** Hybrid Symbolic Execution Process

---

Data: Arbitrary seed

Result: Symbolic

---

```

1:  create struct concrete Input
2:  defined concrete Value
3:  defined operand Index
4:  defined Value sptr
5:  symbolic parameters
6:  create Value Expression
7:  If values is Constant then
8:      get computation's last Instruction
9:      add concrete values to be stored in the symbolic-concrete array
10:   get Symbolic operand () and smt
11:   return symbolic Expressions and save it

```

---

### 3.2.2. Distance-Oriented Path Search Algorithm

Firstly, estimate the distance of the current seed execution state as well as the distance to all successor basic blocks of the current node. Then, execute the closest path, selecting probabilistically if there are multiple paths with the same distance. The distance is calculated based on the number of instructions in the current path. Subsequently, sequentially locate the next target until reaching the final target to conclude the process. This approach effectively reduces overhead and mitigates the problem of path explosion to some extent.

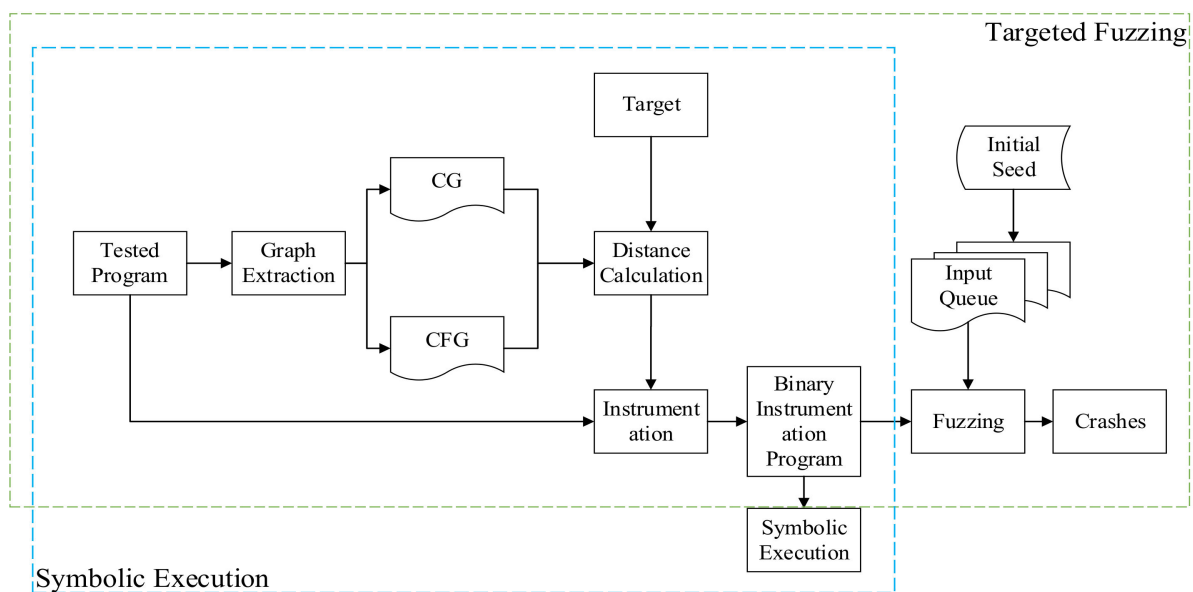
The distance search algorithm proceeds as follows: Heuristically guide the search based on distance. For each execution state  $S$  with  $n$  associated distances corresponding to the distances from  $S$  to  $G_1, \dots, G_{n-1}$ , infer intermediate targets and the final target  $G_n = B$  through static analysis. The closer the intermediate target, the more accurate the distance estimate. Maintain  $n$  priority queues, where the elements in these queues are pointers to execution states  $Q_1, \dots, Q_n$ . These queues provide sorting of the distances from states to their respective targets: the states in front of  $Q_i$  have the shortest estimated distance to target  $Q_i$ , i.e., they are sorted from smallest to largest. In each step of dynamic analysis, select a state  $S$  from the front of one queue, execute instructions at  $S.pc$  using symbolic execution, update the program counter, stack, and address space, and recalculate the distance to the new  $S.pc$ . Gradually advance the states toward the nearest intermediate target using this method, and once  $S.pc = B$  is reached, the search is completed. Finally, generate all the inputs needed for the program to execute along this path.

### 3.3. System Framework

Fuzzing can rapidly generate numerous test cases and trigger some shallow paths. However, it faces challenges in breaking through paths containing magic bytes, making it difficult to reach deeper paths and trigger vulnerabilities at deeper levels. On the other hand, symbolic execution achieves very high coverage and, ideally, can achieve full coverage. Nevertheless, this advantage is counterbalanced by the drawback of path explosion. We address these challenges by combining both techniques and employing directed guidance. This approach effectively mitigates the problem of path explosion in symbolic execution



and addresses the difficulty of fuzzing in exploring deeper vulnerabilities. It enables faster reproduction and the discovery of vulnerabilities and, to some extent, enhances the coverage of the target area. The overall framework of AFL++ is illustrated in Figure 1. For a tested program, graph extraction is first performed to obtain a call graph (CG) and CFG. Then, distance calculation is performed via graph modeling, followed by instrumentation to obtain the binary instrumented program. Subsequently, the process enters the fuzzing's vulnerability discovery module. During fuzzing, the energy allocation and scheduling algorithms allow different path seeds to move towards the target. Symbolic execution is initiated when it is observed that the fuzzing path coverage remains unchanged for a long time or the execution speed is slow. Multiple test cases generated by symbolic execution can trigger deeper path cases to accelerate the speed of fuzzing. The detailed steps are as follows.



**Figure 1.** The overall framework of AFL++.

The first step is the fuzz test stage (core stage). For the program under testing, first perform image extraction to obtain a CG and CFG, then use Python's network module to draw and model the distance. Then, instrumentation is performed, and after obtaining the binary instrumentation program, the stage of fuzzing vulnerability mining is entered. In the fuzzing process, the seeds of different paths move toward the target through the heuristic energy scheduling algorithm. Ideally, start by first triggering shallow paths and digging surface holes. As time goes by after the shallow vulnerabilities are all triggered, the seeds on the deep path will face the vulnerabilities. At this time, the deeper seeds will give more rewards (the probability of triggering the deep vulnerabilities will increase).

The second step is the symbolic execution stage. Symbolic execution is started when it is observed that the fuzzing path coverage remains unchanged for a long time or the execution speed is slow. Symbolic execution has higher path coverage, so fuzzing can increase coverage. Moreover, multiple test cases generated through symbolic execution can trigger deeper paths to speed up fuzzing.

Finally, there is the analysis phase. This phase has been running since the beginning. First, score different test cases through feedback information and judge whether the seed is still "interested" during the fuzzing. If the symbolic execution phase is performed, the "interested" seed is used as the initial input to the symbolic execution phase, and more test cases are generated. Then, the test case execution is scored, where a higher score means a higher chance of fuzzing or symbolic execution in the next input. In addition, test cases are copied to different folders according to whether they increase target coverage or new coverage, crash, hang out, etc.



## 4. Experiment

### 4.1. Experimental Environment

The environment used in the experiments is shown in the following Table 2.

**Table 2.** Experimental environment table.

| Category                | Configuration               |
|-------------------------|-----------------------------|
| Operating system        | Ubuntu 16.04                |
| Kernel version          | 4.15.0                      |
| Core                    | 8                           |
| Memory                  | 16 GB                       |
| Hard drive capacity     | 2TB                         |
| Development environment | AFL2.52b, KLEE, LLVM11.0    |
| Processor               | Inter® Core(TM)i5-10400 USA |

### 4.2. Experimental Design

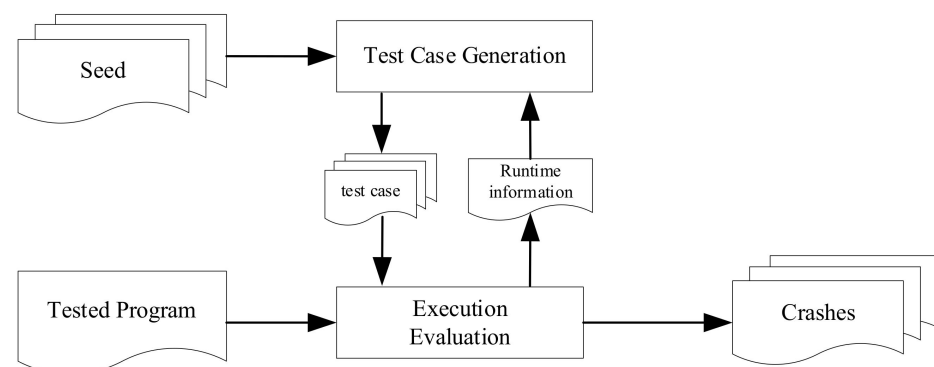
As AFL++ incorporates both symbolic execution and fuzzing techniques, the optimizations implemented by AFL++ can, to some extent, increase the coverage of the target area and execution speed, effectively enhancing vulnerability discovery efficiency. The primary observed metrics include crashes (unique) and total paths, where higher values for both metrics are desirable.

During vulnerability discovery, the main objective is to identify real vulnerabilities, with a focus on observing the value of crashes. If there are no crashes, the coverage rate within a unit of time is observed for comparison. In vulnerability reproduction, the primary goal is the reproduction time, and shorter times for reproducing vulnerabilities are preferable.

Since most vulnerability discovery tools use the same LAVA-M dataset for evaluation, we also utilized this dataset for vulnerability discovery assessment. The LAVA-M dataset consists of four programs: base64, md5sum, uniq, and who, each intentionally injected with multiple vulnerabilities. The number of vulnerabilities in the LAVA-M dataset and the detailed commands are shown in Table 3. The experimental process is shown in Figure 2.

**Table 3.** LAVA-M program detailed data.

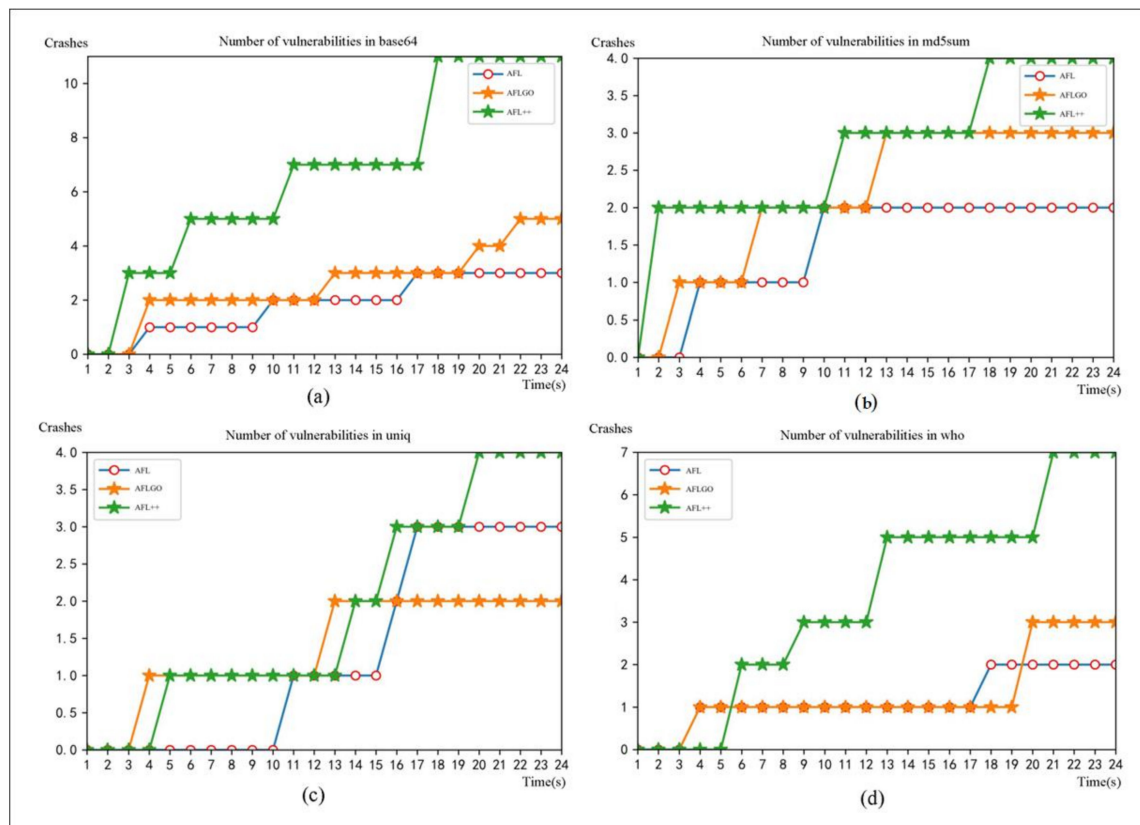
| Program | Vulnerabilities | Complex Command |
|---------|-----------------|-----------------|
| base64  | 44              | ./base64-d@@    |
| md5sum  | 57              | ./md5sum-c@@    |
| uniq    | 44              | ./uniq@@        |
| who     | 2136            | ./who@@         |



**Figure 2.** The experimental process.

### 4.3. Vulnerability Mining Performance Evaluation

Firstly, vulnerability discovery was conducted on LAVA-M. For the purpose of controlling the variables, this study employed the same virtual machine to perform vulnerability discovery on the four programs in LAVA-M using AFL, AFLGo, and AFL++, respectively. The experiment was set to run for 24 h, and the number of crashes discovered by each tool during the process was recorded. The experimental results are presented in Figure 3.



**Figure 3.** Number of vulnerabilities. (a) Number of vulnerabilities in base64. (b) Number of vulnerabilities in md5sum. (c) Number of vulnerabilities in uniq. (d) Number of vulnerabilities in who.

In Figure 3, it can be observed that within 24 h, AFL discovered three crashes on base64, two on md5sum, three on uniq, and two on who. AFLGo found five crashes on base64, three on md5sum, two on uniq, and three on who. AFL++ identified 11 crashes on base64, four on md5sum, four on uniq, and seven on who. It can be seen that for vulnerability mining on the LAVA-M dataset, AFL++ was better. To enhance data visibility, the overall presentation is illustrated in Table 4.

**Table 4.** Number of crashes discovered on LAVA-M within 24 h.

| Method | base64 | md5sum | uniq | who | Total |
|--------|--------|--------|------|-----|-------|
| AFL    | 3      | 2      | 3    | 2   | 10    |
| AFLGo  | 5      | 3      | 2    | 3   | 13    |
| AFL++  | 11     | 4      | 4    | 7   | 26    |

We conducted vulnerability mining evaluations on real programs. The latest version of libxml2 was first selected for vulnerability mining, and programs such as libming, xpdf, libexif, gdb, giflib, jasper, and lrzip were selected. Eight simple programs were tested for 48 h using AFL, AGLGO, and AFL++.

Among the programs, the number of total paths was as follows: AFL had 4818, 5296, 3231, 4209, 1107, 233, 1, 1151; AFLGo had 5812, 5411, 3562, 5579, 1425, 188, 163, 1403; and AFL++ had 7324, 6573, 4517, 8662, 3237, 316, 233, 2282, as shown in Figure 4. Therefore, AFL++ resulted in increases of 52.01%, 24.11%, 39.8%, 110.54%, 192.41%, 35.62%, 233%, and 98.26%, respectively, based on AFL; based on AFLGo, the distribution increases were 26%, 21.47%, 26.81%, 55.26%, 127.15%, 68.08%, 42.94%, and 42.94%, respectively. These data are shown in Table 5. The table clearly shows that the proposed tool outperformed AFLGo and AFL. In Jasper software (<https://github.com/mdadams/jasper>, (accessed on 25 February 2024)), the number of AFL test paths was 1, and the number of tests was 1, but the test with a lower version had 200+ paths. AFL instrumentation likely had a problem, but it did not affect the results of AFL++ and AFLGo, and AFL++ was still better than AFLGo. AFL++ had the most significant advantage in libexif, with more than double that of AFL and AFLGo.

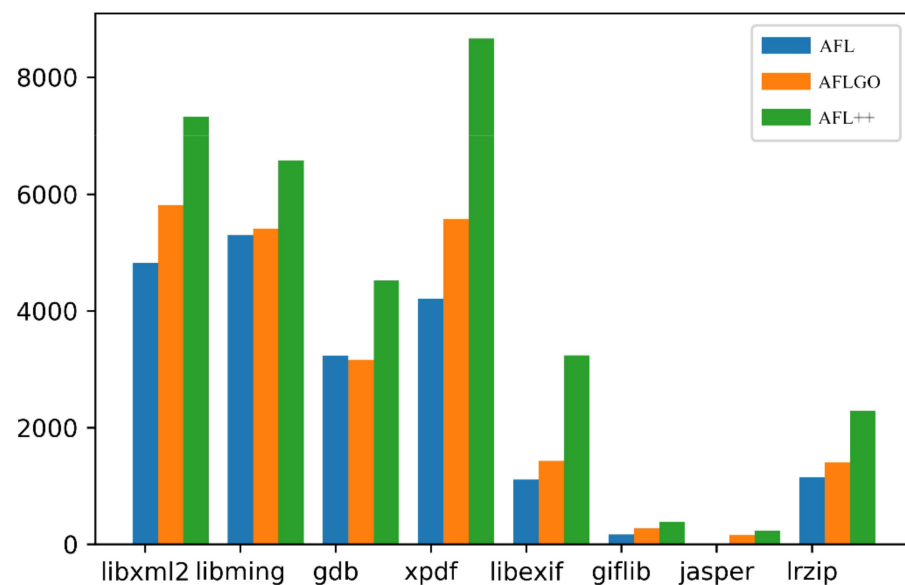


Figure 4. Result comparison chart—total paths.

Table 5. The total paths of AFL, AFLGo, and AFL++ on real programs.

| Method | Libxml2 | Libming | gdb  | xpdf | libexif | giflib | jasper | lrzip |
|--------|---------|---------|------|------|---------|--------|--------|-------|
| AFL    | 4818    | 5296    | 3231 | 4209 | 1107    | 233    | 1      | 1151  |
| AFLGo  | 5812    | 5411    | 3562 | 5579 | 1425    | 188    | 163    | 1403  |
| AFL++  | 7324    | 6573    | 4517 | 8662 | 3237    | 316    | 233    | 2282  |

For crashes, neither AFL, AFLGo, nor AFL++ found crashes on libxml2, libexif, gdb, or jasper. When testing libming, the number of crashes and unique crashes found by AFL++ was 107 (48), which was the most. The crashes were analyzed and submitted, and a CVE was assigned: CVE-2022-44232. For xpdf, the total crashes found by AFL++ were 700 (110), that is, 700 crashes, with a total of 110 unique crashes; the total crashes found by AFLGo were 581 (71); the total crashes found by AFL were 534 (58). The crashes were aggregated into a table, as shown in Table 6. AFL++ found eight vulnerabilities alone, which were analyzed and submitted, and seven CVEs were assigned: CVE-2023-26930, CVE-2023-26931, CVE-2023-26934, CVE-2023-26935, CVE-2023-26936, CVE-2023-26937, and CVE-2023-26938.

**Table 6.** Comparison of quantities of unique crashes.

| Method | Libxml2 | Libming | gdb | xpdf | libexif | giflib | jasper | lrzip |
|--------|---------|---------|-----|------|---------|--------|--------|-------|
| AFL    | 0       | 19      | 0   | 58   | 0       | 0      | 0      | 27    |
| AFLGo  | 0       | 18      | 0   | 71   | 0       | 0      | 0      | 36    |
| AFL++  | 0       | 36      | 0   | 110  | 0       | 3      | 0      | 61    |

Table 6 shows that the number of crashes (unique) in AFL++ had an absolute advantage, especially on giflib, and AFL++ found three path vulnerabilities, but neither AFL nor AFLGo found them.

The above experimental data all show that the vulnerability mining ability of AFL++ is better than that of AFL and AFLGo. This proves that the vulnerability mining framework based on directed fuzzing and symbolic execution can effectively increase the coverage rate and significantly improve the efficiency of vulnerability mining.

#### 4.4. Vulnerability Reproduction Experiment Evaluation

We tested against some open-source programs with public vulnerabilities. We used AFL, AFLGo, and AFL++ to reproduce the vulnerability and evaluate the effectiveness of AFL++ by comparing the time and times of recurrence. Here, we mainly chose binutils and libming to reproduce the vulnerability. Since individual vulnerabilities are difficult to trigger, the time for each test was set to 12 h. To reduce the interference caused by randomness, each experiment was repeated 20 times. The reproduction results of the two vulnerabilities in libming are shown in Table 7. TTE refers to the time from runtime to the first time a vulnerability was triggered, where m is minutes and s is seconds. It can be seen in Table 7 that AFL++ was superior to AFL and AFLGo in the reproduction of these two vulnerabilities.

**Table 7.** Recurrence of vulnerabilities in libming.

| CVE Number | Method | Time | TTE   |
|------------|--------|------|-------|
| 2018-8807  | AFL    | 20   | 13m   |
|            | AFLGo  | 20   | 3m33s |
|            | AFL++  | 20   | 1m55s |
| 2018-8962  | AFL    | 20   | 8m    |
|            | AFLGo  | 20   | 3m21s |
|            | AFL++  | 20   | 1m32s |

In addition, we also conducted experiments on historical vulnerabilities CVE-2016-4487 to CVE-2016-4492 in binutils. The results were compared with currently known targeted fuzzing tools. We used the target definition strategy of CVE-2016-4487 in the AFLGo script to find the target points in the stack backtracking.

Hawkeye was proposed in 2018, and it is also a classic tool in directed greybox fuzzing. AFL-Ant was proposed between 2019 and 2020, and Beacon was proposed in 2022. It is also the latest closed-source directed fuzzing tool. Beacon is based on path pruning. With the assistance of static analysis, all irrelevant paths are deleted, and only the relevant parts are kept to prevent the seeds from executing into branches that have nothing to do with the target code.

It can be seen from Table 8 that, except for the reappearance of CVE-2016-4490, AFL++ took longer than AFL, and the rest were shorter than the AFL reappearance. AFL++ took less time to reproduce than AFLGo, AFL-Ant, and Hawkeye in all the reproduced vulnerabilities. Moreover, for the bugs that take a long time to reproduce on average, although the improved efficiency was not the greatest, the number of recurrences was significantly improved. However, it was still impossible to achieve 20 full recurrences, which may be because the path search algorithm prefers shorter paths rather than full path

coverage, and the randomness of the mutation algorithm resulted in an inability to reach all node paths.

**Table 8.** Recurrence of vulnerabilities in binutils.

| CVE Number | Method  | Time | TTE           |
|------------|---------|------|---------------|
| 2016-4487  | AFL     | 20   | 4m            |
|            | AFLGo   | 20   | 3m            |
|            | Hawkeye | 20   | 2m57s         |
|            | AFL-Ant | 20   | 2m41s         |
|            | Beacon  | -    | 2m31s         |
|            | AFL++   | 20   | <b>2m1s</b>   |
| 2016-4491  | AFL     | 5    | 6h38m         |
|            | AFLGo   | 7    | 5h46m         |
|            | Hawkeye | 9    | 5h12m         |
|            | AFL-Ant | 10   | 6h25m         |
|            | Beacon  | -    | <b>1h23m</b>  |
|            | AFL++   | 15   | 4h47m         |
| 2016-4489  | AFL     | 20   | 7m            |
|            | AFLGo   | 20   | 3m            |
|            | Hawkeye | 20   | 3m26s         |
|            | AFL-Ant | 20   | 3m10s         |
|            | Beacon  | -    | 3m            |
|            | AFL++   | 20   | <b>2m23s</b>  |
| 2016-4492  | AFL     | 20   | 16m           |
|            | AFLGo   | 20   | 9m            |
|            | Hawkeye | 20   | 7m57s         |
|            | AFL-Ant | 20   | 8m51s         |
|            | Beacon  | -    | 6m25s         |
|            | AFL++   | 20   | <b>3m47s</b>  |
| 2016-4490  | AFL     | 20   | <b>59s</b>    |
|            | AFLGo   | 20   | 1m33s         |
|            | Hawkeye | 20   | 1m43s         |
|            | AFL-Ant | 20   | 1m30s         |
|            | Beacon  | -    | 1m22s         |
|            | AFL++   | 20   | 1m27s         |
| 2016-6131  | AFL     | 3    | 7h19m         |
|            | AFLGo   | 5    | 5h53m         |
|            | Hawkeye | 9    | 4h48m         |
|            | AFL-Ant | 7    | 5h35m         |
|            | Beacon  | -    | <b>50m13s</b> |
|            | AFL++   | 12   | 3h31m         |

In the reproduction of CVE-2016-4487, CVE-2016-4489, and CVE-2016-4492, AFL++ took less time than the latest tool, Beacon, and in CVE-2016-4487, it only took 19 s to reproduce at the fastest time. The improvement in CVE-2016-4492 was more prominent, probably because there were multiple paths to reach the target point, and AFL++'s precise guidance and fuzz test random mutation could easily trigger one of the paths. However, in CVE-2016-4491 and CVE-2016-6131, the recurrence time was slightly higher. It may be that the time was too long due to the consumption of a large amount of resources in the AFL++ symbolic execution solution. When reproducing vulnerabilities under shallow paths, AFL++ can find vulnerabilities under shallow paths faster by using symbolic execution solutions due to fewer constraints. AFL++ needs to consume more resources when reproducing deep vulnerabilities due to more constraints. However, Beacon does not need to consider which branches are taken. That is, the mutation can hit the branch no matter what, and the factor of the recurrence time is mainly determined by the mutation algorithm. So, Beacon was slightly faster than AFL++ on deeper paths.

Overall, AFL++ has reasonable practicability and effectiveness in guided vulnerability mining.

## 5. Conclusions

This paper introduces a new vulnerability discovery framework, AFL++, which combines directed greybox fuzzing and symbolic execution techniques. The directed greybox fuzzing technique is improved through the optimization of distance calculation algorithms, energy distribution algorithms, and heuristic energy scheduling algorithms. The symbolic execution technique is enhanced by the hybrid symbolic execution and distance-oriented path search algorithms. The framework's performance is tested in two aspects: vulnerability discovery and vulnerability reproduction. In terms of vulnerability discovery, the LAVA-M dataset and some real-world programs are used for experimental comparison. The results demonstrate that, compared to AFL and AFLGo, AFL++ can effectively increase coverage and improve vulnerability discovery efficiency. AFL++ uncovered vulnerabilities that AFL and AFLGo failed to detect, earning eight CVE numbers, further proving AFL++'s capability in vulnerability discovery. In terms of vulnerability reproduction, using reproduction time for comparison, AFL++ can improve reproduction efficiency compared to AFL, AFLGo, Hawkeye, and AFL-Ant, and also has certain advantages compared to Beacon. These experimental results prove the effectiveness and feasibility of the AFL++ framework.

**Author Contributions:** Conceptualization, G.H.; Methodology, G.H.; Software, G.H. and X.C.; Validation, Y.X. and X.C.; Formal analysis, Y.X. and X.C.; Resources, G.Y.; Data curation, G.Y.; Writing—original draft, G.H.; Writing—review and editing, X.C. and G.Y.; Supervision, G.Y. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** Please check the details through this link: <https://github.com/yxyuestc/AFL++> (accessed on 25 February 2024).

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Böhme, M.; Pham, V.; Nguyen, M.; Roychoudhury, A. Directed greybox fuzzing. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, 30 October–3 November 2017; Thuraisingham, B., Evans, D., Malkin, T., Xu, D., Eds.; ACM: Dallas, TX, USA, 2017; pp. 2329–2344.
2. Cardinale, Y.; Freitas, G.; Valderrama, E.; Aguilera, A.I.; Angsachotmetee, C. Semantic framework of event detection in emergency situations for smart buildings. *Digit. Commun. Networks* **2022**, *8*, 64–79. [\[CrossRef\]](#)
3. Wu, S.; Shen, S.; Xu, X.; Chen, Y.; Zhou, X.; Liu, D.; Xue, X.; Qi, L. Popularityaware and diverse web apis recommendation based on correlation graph. *IEEE Trans. Comput. Soc. Syst.* **2023**, *10*, 771–782. [\[CrossRef\]](#)
4. Mousavi, S.N.; Chen, F.; Abbasi, M.; Khosravi, M.R.; Rafiee, M. Efficient pipelined flow classification for intelligent data processing in iot. *Digit. Commun. Networks* **2022**, *8*, 561–575. [\[CrossRef\]](#)
5. Kim, J.; Yun, J. Poster: Directed hybrid fuzzing on binary code. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, 11–15 November 2019; Cavallaro, L., Kinder, J., Wang, X., Katz, J., Eds.; ACM: London, UK, 2019; pp. 2637–2639.
6. Dong, L.; Li, R. Optimal chunk caching in network coding-based qualitative communication. *Digit. Commun. Networks* **2022**, *8*, 44–50. [\[CrossRef\]](#)
7. Qi, L.; Lin, W.; Zhang, X.; Dou, W.; Xu, X.; Chen, J. A correlation graph based approach for personalized and compatible web apis recommendation in mobile APP development. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 5444–5457. [\[CrossRef\]](#)
8. Dai, H.; Yu, J.; Li, M.; Wang, W.; Liu, A.X.; Ma, J.; Qi, L.; Chen, G. Bloom filter with noisy coding framework for multi-set membership testing. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 6710–6724. [\[CrossRef\]](#)
9. Chen, H.; Xue, Y.; Li, Y.; Chen, B.; Xie, X.; Wu, X.; Liu, Y. Hawkeye: Towards a desired directed grey-box fuzzer. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, 15–19 October 2018; Lie, D., Mannan, M., Backes, M., Wang, X., Eds.; ACM: Toronto, ON, Canada, 2018; pp. 2095–2108.
10. Liang, H.; Zhang, Y.; Yu, Y.; Xie, Z.; Jiang, L. Sequence coverage directed greybox fuzzing. In Proceedings of the 27th International Conference on Program Comprehension, ICPC 2019, Montreal, QC, Canada, 25–31 May 2019; Gu'eh'eneuc, Y., Khomh, F., Sarro, F., Eds.; IEEE/ACM: Montreal, QC, Canada, 2019; pp. 249–259.
11. Zheng, Y.; Li, Z.; Xu, X.; Zhao, Q. Dynamic defenses in cyber security: Techniques, methods and challenges. *Digit. Commun. Networks* **2022**, *8*, 422–435. [\[CrossRef\]](#)



12. Wang, F.; Wang, L.; Li, G.; Wang, Y.; Lv, C.; Qi, L. Edge-cloud-enabled matrix factorization for diversified apis recommendation in mashup creation. *World Wide Web* **2022**, *25*, 1809–1829. [\[CrossRef\]](#)
13. Li, J.; Luo, X.; Zhang, Y.; Zhang, P.; Yang, C.; Liu, F. Extracting embedded messages using adaptive steganography based on optimal syndrome-trellis decoding paths. *Digit. Commun. Networks* **2022**, *8*, 455–465. [\[CrossRef\]](#)
14. Liang, H.; Jiang, L.; Ai, L.; Wei, J. Sequence directed hybrid fuzzing. In Proceedings of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2020, London, ON, Canada, 18–21 February 2020; Kontogiannis, K., Khomh, F., Chatzigeorgiou, A., Fokaefs, M., Zhou, M., Eds.; IEEE: London, ON, Canada, 2020; pp. 127–137.
15. Osterlund, S.; Razavi, K.; Bos, H.; Giuffrida, C. Parmesan: Sanitizer-guided greybox fuzzing. In Proceedings of the 29th USENIX Security Symposium, USENIX Security 2020, Boston, MA, USA, 12–14 August 2020; Capkun, S., Roesner, F., Eds.; USENIX Association: Boston, MA, USA, 2020; pp. 2289–2306.
16. Zhu, X.; Liu, S.; Li, X.; Wen, S.; Zhang, J.; Camtepe, S.A.; Xiang, Y. Defuzz: Deep learning guided directed fuzzing. *arXiv* **2020**, arXiv:2010.12149.
17. Zhao, J. Constructing more complete control flow graphs utilizing directed graybox fuzzing. *Appl. Sci.* **2021**, *11*, 1351. [\[CrossRef\]](#)
18. Lee, G.; Shim, W.; Lee, B. Constraint-guided directed greybox fuzzing. In Proceedings of the 30th USENIX Security Symposium, USENIX Security 2021, Vancouver, BC, Canada, 11–13 August 2021; Bailey, M., Greenstadt, R., Eds.; USENIX Association: Vancouver, BC, Canada, 2021; pp. 3559–3576.
19. Zhu, X.; Böhme, M. Regression greybox fuzzing. In Proceedings of the CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, 15–19 November 2021; Kim, Y., Kim, J., Vigna, G., Shi, E., Eds.; ACM: Icheon-si, Republic of Korea, 2021; pp. 2169–2182.
20. Wang, S.; Jiang, X.; Yu, X.; Sun, S. Kcfuzz: Directed fuzzing based on keypoint coverage. In Proceedings of the Artificial Intelligence and Security—7th International Conference, ICAIS 2021, Dublin, Ireland, 19–23 July 2021; Proceedings, Part I; Lecture Notes in Computer Science. Sun, X., Zhang, X., Xia, Z., Bertino, E., Eds.; Springer: Dublin, Republic of Ireland, 2021; Volume 12736, pp. 312–325.
21. Pham, V.; Nguyen, M.; Ta, Q.; Murray, T.; Rubinstein, B.I.P. Towards systematic and dynamic task allocation for collaborative parallel fuzzing. In Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, 15–19 November 2021; IEEE: Melbourne, Australia, 2021; pp. 1337–1341.
22. Huang, H.; Guo, Y.; Shi, Q.; Yao, P.; Wu, R.; Zhang, C. BEACON: Directed greybox fuzzing with provable path pruning. In Proceedings of the 43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, 22–26 May 2022; IEEE: San Francisco, CA, USA, 2022; pp. 36–50.
23. Du, Z.; Li, Y.; Liu, Y.; Mao, B. Windranger: A directed greybox fuzzer driven by deviation basic blocks. In Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, 25–27 May 2022; pp. 2440–2451.
24. Canakci, S.; Matyunin, N.; Graffi, K.; Joshi, A.; Egele, M. Targetfuzz: Using darts to guide directed greybox fuzzers. In Proceedings of the ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May–3 June 2022; Suga, Y., Sakurai, K., Ding, X., Sako, K., Eds.; ACM: Nagasaki, Japan, 2022; pp. 561–573.
25. Sen, K. DART: Directed automated random testing. In Proceedings of the Hardware and Software: Verification and Testing—5th International Haifa Verification Conference, HVC 2009, Haifa, Israel, 19–22 October 2009; Revised Selected Papers. Lecture Notes in Computer Science. Namjoshi, K.S., Zeller, A., Ziv, A., Eds.; Springer: Haifa, Israel, 2009; Volume 6405, p. 4.
26. Ma, K.; Khoo, Y.P.; Foster, J.S.; Hicks, M. Directed symbolic execution. In Proceedings of the Static Analysis—18th International Symposium, SAS 2011, Venice, Italy, 14–16 September 2011; Proceedings. Lecture Notes in Computer Science. Yahav, E., Ed.; Springer: Venice, Italy, 2011; Volume 6887, pp. 95–111.
27. Rustamov, F.; Kim, J.; Yu, J.; Kim, H.; Yun, J. Bugminer: Mining the hard-to-reach software vulnerabilities through the target-oriented hybrid fuzzer. *Electronics* **2020**, *10*, 62. [\[CrossRef\]](#)
28. Poeplau, S.; Francillon, A. Symbolic execution with symcc: Don't interpret, compile! In Proceedings of the 29th USENIX Security Symposium, USENIX Security 2020, Boston, MA, USA, 12–14 August 2020; Capkun, S., Roesner, F., Eds.; USENIX Association: Boston, MA, USA, 2020; pp. 181–198.
29. Poeplau, S.; Francillon, A. Symqemu: Compilation-based symbolic execution for binaries. In Proceedings of the 28th Annual Network and Distributed System Security Symposium, NDSS 2021, Virtually, 21–25 February 2021; The Internet Society: Reston, VA, USA, 2021.

**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.